

MobilePlantViT-LDA: A Lightweight Hybrid CNN-Transformer Model for Plant Leaf Disease Detection

M. Deepika
Assistant Professor
Department of CSE(AIML)
Technology Neil Gogte Institute of Technology
Hyderabad, India
deepikamoksh2004@gmail.com

Vaishnavi Abbanaboina
Student, CSE
Neil Gogte Institute of Technology
Hyderabad, India
vaishnavi.abbanaboina21@gmail.com

Sangineedi Chaitanya Satya Balaji
Student, CSE
Neil Gogte Institute of
Hyderabad, India
chaitanyasatyabalaji@gmail.com

Gangiseti Himasree
Student, CSE
Neil Gogte Institute of Technology
Hyderabad, India
himarm2006@gmail.com

Astha Kumari
Student, CSE
Neil Gogte Institute of Technology
Hyderabad, India
009asthakumari@gmail.com

Sreeja Vaitla
Student, CSE
Neil Gogte Institute of Technology
Hyderabad, India
sreejavaitla@gmail.com

Abstract—This paper presents MobilePlantViT-LDA, an intelligent mobile-based plant leaf disease detection system that accurately classifies diseases and recommends the appropriate treatment to farmers. The system integrates image preprocessing, lightweight convolutional feature extraction, attention enhancement, transformer-based encoding, and a classification layer to predict disease types with confidence scores. Achieving 97.48% accuracy in the PlantVillage dataset, the model outperforms the baseline networks while maintaining efficiency for mobile deployment. The source code is available in the GitHub repository: <https://github.com/SCSBalaji/MobilePlantViT-LDA>

Index Terms—plant disease detection, deep learning, vision transformer, mobile deployment, convolutional neural networks, attention mechanism

I. INTRODUCTION

This paper focuses on the design of an intelligent mobilebased plant leaf disease detection system that can accurately classify diseases and recommend appropriate treatment to farmers. The pipeline consists of image preprocessing, lightweight convolutional feature extraction, attention enhancement, transformer-based encoding, and a classification layer to predict the disease type along with confidence score integrated in it.

A. Project Overview

Plant diseases threaten global agriculture, causing economic losses and food insecurity. The FAO reports that nearly 40% of global crop production is lost annually to plant diseases. Timely, accurate disease identification is vital for sustainable agriculture and food security.

Recent progress in deep learning has made it possible to automatically recognize plant diseases from images [1]. Convolutional Neural Networks (CNNs) have been successful,

but they mainly extract local features and often miss broader patterns in plant images. Vision Transformers (ViTs) address this by capturing global spatial relationships, which helps improve classification accuracy [2].

Traditional ViT models are effective but require extensive computational and memory resources. They are difficult to run on devices with limited power or memory, such as mobile phones or edge devices. To address these issues, this work proposes MobilePlantViT-LDA. It is a lightweight hybrid Vision Transformer architecture optimized for efficient plant disease classification [3].

The proposed model employs Ghost Convolutions for efficient feature extraction, Coordinate Attention to emphasize important disease regions, and Fused-Inverted Residual Blocks to enhance feature representation. Linear Differential Attention is incorporated to reduce computational complexity, while a Bottleneck Feed Forward Network (FFN) maintains a compact model size without sacrificing accuracy. Together, these components enable MobilePlantViT-LDA to achieve a balance between accuracy, speed, and efficiency, making it suitable for deployment on low-power devices in real-world agricultural environments.

B. Background and Motivation

The identification of plant diseases has traditionally relied on manual inspection by agricultural experts, a process that is time-consuming, subjective, and often inaccurate under field conditions. The introduction of artificial intelligence and computer vision has revolutionized this domain by enabling rapid and objective disease recognition. However, most state-of-the-art deep learning models remain computationally intensive, limiting their practicality for on-site diagnosis,

particularly in rural regions where computational resources are limited. Vision Transformers have demonstrated exceptional capabilities in visual understanding by leveraging self-attention mechanisms. Nevertheless, their high computational overhead poses a barrier to real-time applications. There is thus a growing need for lightweight ViT architectures that retain the accuracy of transformer-based models while minimizing resource consumption. This research is motivated by the need to design an optimized ViT that maintains high accuracy in disease detection while remaining computationally lightweight. MobilePlantViT-LDA addresses this challenge using simple architecture, hybrid feature learning, and parameter reduction, thereby enabling the deployment of AI-driven plant disease recognition systems in practical agricultural settings.

II. LITERATURE SURVEY

A. Introduction to ViT

Vision Transformers (ViTs) are an important development in computer vision as they introduce a new way for machines to process images. Inspired by the transformer models used in natural language processing, ViTs apply attention mechanisms to visual data instead of text. Rather than using convolutional layers like traditional CNNs, they divide an image into small patches, treat these patches as a sequence, and pass them through a transformer model to learn how different parts of the image relate to each other. A key advantage of ViTs is their self-attention mechanism. This allows the model to understand connections between distant regions in an image. In contrast, convolutional neural networks focus mainly on local areas within a fixed receptive field. Because ViTs can analyze multiple regions simultaneously, they are better at combining fine details with global image context. This leads to strong performance in tasks such as image classification, segmentation, and object detection.

Vision Transformers (ViTs) are typically trained in two stages. First, they are pre-trained on large-scale image datasets such as ImageNet or JFT, where they learn general visual patterns and high-level feature representations. This stage provides a strong foundation for understanding visual structures across different types of images. In the second stage, the model is fine-tuned for a specific task which allows the transformer to adjust its learned representations to match the requirements of applications such as image classification, object detection, or segmentation. Because ViTs can capture global relationships across the entire image through self-attention, they often achieve higher accuracy and show greater adaptability than traditional convolutional neural networks (CNNs). However, this improved performance comes with a trade-off between accuracy and efficiency. While ViTs deliver strong results, they require substantial processing power and memory, which limits their deployment on low-power or mobile devices. To overcome this issue, lightweight and hybrid architectures such as MobileViT, EfficientFormer, and MobilePlantViT were introduced. These models integrate convolutional layers with transformer blocks, combining the efficiency of CNNs with the global modeling capability of transformers.

As a result, Vision Transformers are now being applied not only in conventional computer vision tasks but also in specialized domains such as plant disease detection, medical imaging, and remote sensing, where capturing subtle visual variations is essential.

B. Existing Works

Kai Han et al. [1] propose GhostNet, a lightweight CNN that decomposes feature generation into intrinsic and ghost feature maps using cheap linear operations. This significantly reduces parameters and FLOPs compared to standard convolutions while maintaining competitive accuracy, achieving 75.7% top1 on ImageNet.

Qibin Hou et al. [2] propose Coordinate Attention, which enhances channel attention by decomposing global pooling into two 1D encodings along height and width to embed precise positional information. This enables the network to capture long-range dependencies while retaining spatial awareness, improving top-1 accuracy by 0.8% on ImageNet.

Li et al. [3] introduce EfficientFormer, a hybrid vision transformer that combines Fused-Inverted Residual blocks with linear attention to achieve MobileNet-level efficiency while retaining global feature extraction. Token reduction across stages further lowers computational cost, achieving 79.2% top1 accuracy on ImageNet.

Xu et al. [4] propose Linear Differential Attention (LDA), which enhances traditional linear attention using differential kernels to capture complex feature interactions while maintaining $O(N)$ complexity. LDA outperforms prior linear transformers such as Performer and Linear Transformer by up to 1.8% on ImageNet-1K and CIFAR benchmarks.

Liu et al. [5] propose ConvNeXt V2, which bridges traditional ConvNets and vision transformers by integrating Bottleneck FFNs with masked autoencoder pretraining to improve learning efficiency. Inverted bottleneck designs and improved normalization reduce computational cost while achieving 86.8% top-1 accuracy on ImageNet-1K.

Mehta and Rastegari [6] introduce MobileViT, a hybrid architecture that replaces heavy convolutional layers with transformer blocks to capture long-range dependencies while preserving local features, achieving 74.8% top-1 accuracy on ImageNet with multi-head self-attention over non-overlapping patches.

Mehta and Rastegari [7] further propose MobileViTv2, introducing Separable Linear Self-Attention that splits key and value projections into distinct linear transformations, reducing parameters and latency while maintaining high accuracy. MobileViTv2-S achieves 78.1% top-1 on ImageNet-1K.

Ullah et al. [8] propose AppViT, a lightweight vision transformer for apple leaf disease detection that combines depthwise convolutions with reduced-dimension attention layers to balance local and global feature learning, achieving over 96% accuracy with low computational overhead.

Borhani et al. [9] apply a standard Vision Transformer to automated plant disease classification, demonstrating that selfattention mechanisms effectively capture global leaf

patterns missed by CNNs. The model achieves over 99% accuracy on benchmark plant disease datasets.

Kumar and Arya [10] propose PDLCViT, a patch-based ViT encoder-decoder framework that performs simultaneous disease classification and lesion localization without boundingbox annotations. Class activation maps identify affected leaf regions, achieving 98.3% classification accuracy on plant disease datasets.

III. PROPOSED WORK

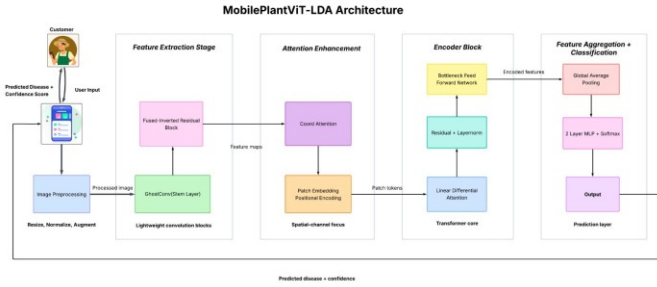


Fig. 1: Architecture Diagram of the Proposed Work

A. Data Collection

The PlantVillage dataset was selected for training the proposed model. The dataset mainly consists of high-resolution RGB images which are more than 54,000 images of plant leaves collected from 26 different plant species. These images include both healthy leaves and diseased leaves. After preprocessing, the final refined dataset comprises 49,370 images, removing 9.1% of corrupted or duplicate samples to ensure training data quality.

TABLE I: Dataset Configuration

Property	Value
Source	PlantVillage (Kaggle) [11]
Original Size	54,304 images
Classes	38
Preprocessing	Corruption check & pHash deduplication
Final Refined Size	49,370 images
Train (70.0%)	34,539 images
Val (15.0%)	7,388 images
Test (15.1%)	7,443 images

The refined dataset distribution across plant categories is shown in Table II, highlighting the class imbalance that reflects real-world agricultural prevalence patterns. This imbalance necessitates careful validation strategies and data augmentation to prevent model bias toward dominant classes. The dataset was partitioned into training (70%), validation (15%), and test (15%) sets using stratified sampling to preserve class distribution across all 38 classes. This dataset supports the model in identifying whether a leaf is healthy or diseased. The PlantVillage dataset was obtained from the Kaggle platform.

TABLE II: PlantVillage dataset distribution after preprocessing. Stratified 70/15/15 split applied per class.

Category	Classes	Total Images
Apple	4	3,085
Blueberry	1	1,408
Cherry	2	1,786
Corn(Maize)	4	3,759
Grape	4	3,696
Orange	1	4,811
Peach	2	2,530
Pepper Bell	2	2,342
Potato	3	2,062
Raspberry	1	298
Soybean	1	3,918
Squash	1	1,818
Strawberry	2	1,464
Tomato	10	16,393
Total	38	49,370

B. Preprocessing and Augmentation

The main preprocessing method used was image resizing, normalization and augmentation where all images were normalized using the standard ImageNet statistics:

- Mean: (0.485, 0.456, 0.406)
- Standard Deviation: (0.229, 0.224, 0.225)

Channel-wise normalization standardizes pixel values independently across R, G, and B channels, reducing illumination and contrast variations common in field-captured plant images. By centering each channel to zero mean and unit variance, it stabilizes gradient flow during optimization, prevents training divergence from varying lighting conditions, and aligns input distributions with pretrained visual backbones like those in MobileNet and EfficientNet families. This preprocessing ensures the model focuses on disease-specific color, texture, and lesion patterns rather than irrelevant photometric differences across images.

Augmentation was performed to artificially increase dataset diversity and improve model robustness to real-world farm conditions as shown in Table III.

TABLE III: Data Augmentation

Augmentation	Parameters
Horizontal flip	50%
Random rotation	$\pm 30^\circ$
Random crop	224×224 from 256
Affine transform	trans.=0.1, scale=(0.9,1.1), shear=10
Color jitter	bright.=0.2, cont.=0.2, sat.=0.2, hue=0.1
Random perspective	20% distortion, $p=0.5$
Normalization	$\mu=[0.485,0.456,0.406]$, $\sigma=[0.229,0.224,0.225]$

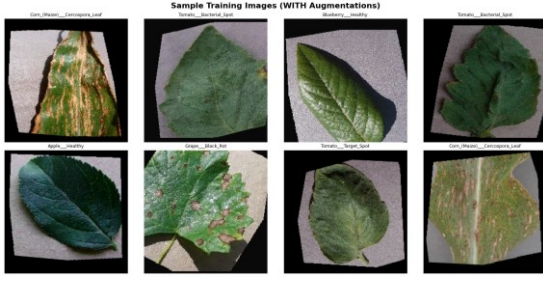


Fig. 2: Sample Training Images (WITH Augmentations) showing Corn cercospora leaf, Tomato bacterial spot, Blueberry healthy, Tomato bacterial spot, Apple healthy, Grape black rot, Tomato target spot and Corn cercospora leaf.

These transformations help the model generalize better by simulating natural leaf appearance variations such as different camera angles, lighting changes, scaling and orientation differences, partial occlusion, and background disturbance.

The sample images represent multiple plant species and disease types, including: Corn (Cercospora Leaf Spot), Tomato (Bacterial Spot / Target Spot), Grape (Black Rot), Apple (Healthy), Blueberry (Healthy).

IV. METHODOLOGY

A. Feature Extraction Stage

The Feature Extraction Stage acts as the backbone of the model and operates directly on the preprocessed input tensor $x_0 \in \mathbb{R}^{H \times W \times 3}$, where the image has already been resized and normalized. Its primary goal is to transform this input into a hierarchy of rich, compressed feature maps that capture visual patterns such as edges, textures, and high-level shapes while progressively reducing spatial resolution and increasing channel depth.

1) *GhostConv (Stem Layer)*: The Stem layer acts as the initial feature extractor and performs the first spatial downsampling while expanding the channel dimension. Instead of using a standard convolution, it employs a Ghost module that decomposes feature generation into intrinsic and inexpensive ghost features. Formally, the stem produces an intrinsic feature map

$$U = W_s * x_0, \quad U \in \mathbb{R}^{sH \times sW \times C_{int}} \quad (1)$$

where W_s is a standard convolution kernel, s denotes the stride, and C_{int} represents the number of intrinsic channels. Cheap linear operations $\{\phi_k(\cdot)\}_{k=1}^K$ (e.g., depthwise filters or spatial shifts) are then applied to generate additional ghost feature maps:

$$G_k = \phi_k(U), \quad k = 1, \dots, K. \quad (2)$$

These feature maps are concatenated with the intrinsic features to form the final stem output feature map:

$$F_{stem} = \text{Concat}(U, G_1, \dots, G_K). \quad (3)$$

This mechanism significantly reduces the number of parameters and floating-point operations (FLOPs) compared to a single dense convolution that directly produces the same number of output channels.

2) *Fused-Inverted Residual Blocks*: After the stem stage, the feature tensor F_{stem} is passed through a lightweight convolutional block implemented as a *Fused-Inverted Residual (FIR)* unit, corresponding to the *Fused-Inverted Residual Block* shown in the architecture diagram. The inverted residual design follows a Narrow $\rightarrow$ Wide $\rightarrow$ Narrow transformation, where a low-dimensional input is first expanded, spatially processed, and then projected back to a compact bottleneck representation.

Given an input feature map $x \in \mathbb{R}^{H \times W \times C_{in}}$, the FIR block can be formulated as

$$z = \sigma(\text{BN}(W_{fused} * x)), \quad (4)$$

$$y = W_{proj} * z, \quad (5)$$

where W_{fused} denotes a single 3×3 convolution that jointly performs channel expansion and spatial feature extraction, W_{proj} represents a 1×1 convolution projecting the features to C_{out} channels, $\sigma(\cdot)$ is a nonlinear activation function (e.g., SiLU), and BN denotes batch normalization.

When $C_{in} = C_{out}$ and the stride equals 1, a residual connection is employed to facilitate information preservation and stable gradient propagation, yielding

$$y = x + z. \quad (6)$$

The resulting output feature map $F_{FIR} = y$ is forwarded directly to the subsequent Attention Enhancement module, consistent with the single forward connection from the Fused-Inverted Residual Block to the Coordinate Attention stage in the architecture diagram. The output feature map from each block is forwarded to the Attention Enhancement stage which enables multi-scale feature representation across spatial resolutions.

B. Attention Enhancement Stage

This module serves as a bridge between local CNN representations and the global Transformer encoder by operating directly on the output feature map of the Fused-Inverted Residual block. It refines the extracted features using Coordinate Attention, which enhances channel attention by embedding explicit positional information, enabling the network to attend to both *what* features are important and *where* disease-related cues appear on the leaf surface.

Let the input feature map be $F_{FIR} \in \mathbb{R}^{H \times W \times C}$. Coordinate Attention first applies two independent one-dimensional global pooling operations along the horizontal and vertical spatial directions:

$$f_h(c, x) = \frac{1}{H'} \sum_{y=1}^{H'} F_{FIR}(y, x, c), \quad (7)$$

$$f_v(c, y) = \frac{1}{W'} \sum_{x=1}^{W'} F_{FIR}(y, x, c), \quad (8)$$

which generate two context descriptors $f_h \in \mathbb{R}^{1 \times W \times C}$ and $f_v \in \mathbb{R}^{H \times 1 \times C}$ that preserve spatial information along a single axis.

These descriptors are passed through shared transformation layers and then separated into horizontal and vertical attention maps, denoted as a_h and a_v . The attention maps are subsequently broadcast and applied to the original feature map as

$$\tilde{F}(y, x, c) = F_{\text{FIR}}(y, x, c) \cdot a_h(c, x) \cdot a_v(c, y), \quad (9)$$

resulting in a refined feature tensor $\tilde{F}$, where channel responses are adaptively reweighted based on their spatial importance.

In the Patch Embedding and Positional Encoding stage, the refined feature map $\tilde{F}$ is converted into a sequence of patch tokens suitable for transformer processing. The feature map is partitioned into non-overlapping patches of size $P \times P$. Each patch is flattened and linearly projected using a learnable embedding matrix $W_p \in \mathbb{R}^{(P^2 C) \times D}$, producing patch-level tokens:

$$z_i = W_p \text{vec}(\tilde{F}^{(i)}), \quad z_i \in \mathbb{R}^D, \quad i = 1, \dots, N, \quad (10)$$

where $N = \frac{H'W'}{P^2}$ denotes the total number of patches and D is the token embedding dimension.

To preserve spatial ordering information lost during flattening, a learnable positional encoding $p_i \in \mathbb{R}^D$ is added to each token, yielding the initial transformer input sequence:

$$x_i^{(0)} = z_i + p_i, \quad i = 1, \dots, N. \quad (11)$$

C. Transformer Core (Encoder Block)

The transformer core captures long-range global dependencies among patch tokens that are difficult to model using convolutional operations alone. It is implemented as a lightweight transformer encoder composed of Linear Differential Attention (LDA), residual connections with layer normalization, and a bottleneck Feed Forward Network (FFN) optimized for mobile efficiency.

Given the input token sequence

$$X^{(0)} = [x_1^{(0)}, \dots, x_N^{(0)}] \in \mathbb{R}^{N \times D}, \quad (12)$$

each encoder layer produces an updated representation $X^{(l)}$ by sequentially applying LDA-based self-attention and a bottleneck FFN. Residual connections and layer normalization are employed around both sublayers to stabilize training and maintain effective information flow.

1) *Linear Differential Attention*: In standard Transformer architectures such as Vision Transformers (ViT), the selfattention mechanism is defined as

$$\text{Att}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad (13)$$

where $Q, K, V \in \mathbb{R}^{N \times d_k}$ represent the query, key, and value matrices, respectively. This formulation incurs a computational complexity of $O(N^2)$ with respect to the number of patches N , which is impractical for deployment on resourceconstrained edge devices.

Linear Differential Attention reformulates self-attention into a linear $O(N)$ operation while enhancing discriminative capability through a differential mechanism. First, the query and key matrices are transformed using a kernel mapping function $\phi(\cdot)$:

$$Q^\sim = \phi(Q), \quad K^\sim = \phi(K). \quad (14)$$

This enables attention computation in a factored linear form:

$$\text{LinAtt}(Q, K, V) = \tilde{Q} \left(\tilde{K}^\top V \right), \quad (15)$$

which avoids explicit construction of the $N \times N$ similarity matrix.

To further emphasize informative disease-related patterns, LDA computes two linear attention maps with distinct parameterizations, LinAtt_A and LinAtt_B , and combines them through subtraction:

$$\text{LDA}(Q, K, V) = \text{LinAtt}_A(Q, K, V) - \text{LinAtt}_B(Q, K, V). \quad (16)$$

This differential formulation suppresses common background responses while amplifying salient disease-specific features.

Together, kernel-based linearization and differential attention reduce the computational complexity from $O(N^2)$ to $O(N)$ and improve robustness to noise, enabling accurate and efficient global modeling of plant disease patterns on mobile and edge hardware.

Working Mechanism:

In the first step, Dual attention generation is used where instead of generating one attention map, the LDA block generates two separate linear attention maps.

Attention_{LDA}

$$(Q, K, V) = \frac{\phi(Q_{\lambda_1}) (K_{\lambda_1}^\top V)}{\phi(Q_{\lambda_1}) \sum K_{\lambda_1}^\top} - \lambda \frac{\phi(Q_{\lambda_2}) (K_{\lambda_2}^\top V)}{\phi(Q_{\lambda_2}) \sum K_{\lambda_2}^\top} \quad (17)$$

Where: $\phi(\cdot)$ is the kernel function (typically $\text{ELU}(x) + 1$) used to linearize the complexity from $O(N^2)$ to $O(N)$. The λ parameter represents the differential scaling factor that helps cancel out common background noise. In the third step, irrelevant background information (like the sky or soil in a plant image) usually appears similarly in both maps. When you subtract them ($A-B$), this common noise cancels out or becomes close to zero.

TABLE IV: Comparison of LDA over other mechanisms

Feature	Standard	Linear	LDA
Complexity	High (N^2)	Low (N)	Low (N)
Focus Ability	High	Low	Very High
Noise Handling	Poor	Average	Excellent
Best For	GPUs/Servers	Mobile Tasks	Edge Devices

2) *Residual Connections and Layer Normalization*: In deep networks, gradients (learning signals) often get smaller and smaller as they travel back from the output to the input, eventually reaching zero (vanishing). The Residual connection creates a path for gradients to flow backward smoothly, allowing the earlier layers (like GhostConv stem) to learn effectively. This is done using:

$$Y = \text{Concat}([X \cdot f_{\text{primary}}, \Phi_{i,j}(y_i)]) \quad (18)$$

Where: y'_i is the i -th intrinsic feature map generated by a standard convolution $f_{primary}$, $\Phi_{i,j}$ represents the j -th linear transformation (e.g., a 3×3 depthwise convolution) used to generate the "ghost" maps.

Layer Normalization ensures that the inputs to the subsequent Feed Forward Network (FFN) lie within a stable and consistent range by normalizing features across the hidden dimension for each token independently. For a given token, it computes the mean μ and variance σ^2 of the feature activations and applies normalization as follows:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta, \quad (19)$$

where x denotes the input activation, μ and σ^2 are the mean and variance of the input features, respectively, ϵ is a small constant added for numerical stability, and γ and β are learnable scale and shift parameters.

In the attention sub-layer, Layer Normalization is applied after the residual connection, yielding

$$\text{Output} = \text{LayerNorm}(\text{Attention}(x) + x), \quad (20)$$

which corresponds to the residual-plus-normalization structure illustrated after the Linear Differential Attention module in the architecture diagram.

3) *Bottleneck Feed Forward Network*: Standard transformer FFNs expand the input dimension d to a large hidden size (typically $4d$), apply nonlinearity, then project back to d , incurring high parameter cost unsuitable for mobile deployment. The Bottleneck FFN adopts a reduced expansion ratio

($d_{\text{bottleneck}} = 2d$) that substantially lowers parameter count from $8d^2$ to $4d^2$ while preserving nonlinear modeling capacity. Formally, given input tokens $X \in \mathbb{R}^{N \times d}$, the operation is:

$$\text{FFN}(x) = W_2 \sigma(W_1 x), \quad (21)$$

where $W_1 \in \mathbb{R}^{d \times 2d}$, $W_2 \in \mathbb{R}^{2d \times d}$, and σ is SiLU activation. This bottleneck design substantially reduces parameter count and computational overhead while retaining sufficient nonlinear modeling capacity for accurate plant disease classification on mobile and edge devices.

TABLE V: Comparison of Bottleneck over Standard FFNs

Feature	Standard FFN	Bottleneck FFN
Parameter Count	High	Low
Computational cost	Expensive	Cheap
Overfitting Risk	High on small datasets	Low
Performance	Great for servers	Great for mobile

D. Algorithm: Hybrid CNN–Attention–Transformer Model

1. Input Stage – User and Image Preprocessing

Input: An input image is provided (e.g., a leaf or crop photograph) through the mobile application interface. Steps:

- Receive the raw RGB image from the user interface and convert it into a tensor $x_0 \in \mathbb{R}^{H \times W \times 3}$.
- Preprocess the image by resizing it to a fixed resolution (e.g., 224×224), normalizing pixel values to a standard

range, and optionally applying data augmentation techniques (rotation, flipping, color jitter, etc.) during training.

- Output a preprocessed image tensor x'_0 that is ready for feature extraction.

2. Feature Extraction Stage – GhostConv Stem and Fused-Inverted Residual Block

Goal: Extract informative convolutional features in a lightweight manner before passing them to the Coordinate Attention module. Process:

- Pass x'_0 through the GhostConv (Stem Layer), which performs initial downsampling and channel expansion to produce the stem feature map F_{stem} .
- Feed F_{stem} into a single Fused-Inverted Residual Block, which expands channels, applies a fused 3×3 convolution for spatial feature extraction, and projects the features back to a bottleneck representation, yielding F_{FIR} .
- Forward F_{FIR} directly to the Attention Enhancement Stage (Coordinate Attention) as the sole convolutional feature map output of this stage, consistent with the MobilePlantViT-LDA architecture diagram.

3. Attention Enhancement – Coordinate Attention Module

Goal: Enhance the most informative spatial and channelwise features in the single convolutional feature map produced by the feature extraction stage. Steps:

- Feed the feature map F_{FIR} from the Fused-Inverted Residual Block into the Coordinate Attention module, which performs separate one-dimensional global pooling operations along the horizontal and vertical directions to preserve positional information.
- Coordinate Attention decomposes channel attention into spatially aware coordinate embeddings and generates horizontal and vertical attention maps that reweight the original feature map.
- The output is a refined, attention-enhanced feature map $\tilde{F}$ that emphasizes disease-relevant regions on the leaf while suppressing less informative background responses.

4. Patch Embedding and Positional Encoding

Goal: Convert the attention-refined spatial feature map into a sequence of patch tokens suitable for transformer-based processing. Steps:

- Partition the attention-enhanced feature map $\tilde{F}$ into nonoverlapping patches (e.g., 16×16) across the spatial dimensions.
- Flatten each patch and project it through a learnable linear embedding layer to obtain a token representation of dimension D , forming a sequence of N patch tokens.
- Add learnable positional encodings to each token to preserve the original spatial ordering before passing the sequence to the transformer encoder.

5. Encoder Block (Transformer Core)

Goal: Capture long-range dependencies between patch tokens in a mobile-efficient manner. Steps:

- Feed the patch token sequence into the Linear Differential Attention layer to compute self-attention with linear complexity while enhancing discriminative disease-related patterns.
- Apply a Residual Connection followed by Layer Normalization around the attention output to stabilize training and maintain effective information flow.
- Pass the normalized representations through a Bottleneck Feed Forward Network (FFN) to refine token features, followed again by a residual connection and layer normalization as part of the transformer encoder stack.

6. Feature Aggregation + Classification

Goal: Aggregate encoded features and predict class probabilities.

Steps:

- Apply Global Average Pooling (GAP) across the encoded feature map.
- Pass the pooled vector into a Linear Layer + Softmax classifier.
- Output the Predicted Disease Class and Confidence Score.

The hyperparameters used during training were selected through a validation-guided empirical tuning process. Several learning rate values in the range of 1×10^{-4} to 5×10^{-4} were evaluated, and a learning rate of 2×10^{-4} (0.0002) provided the most stable convergence and highest validation accuracy. Weight decay was introduced to reduce overfitting and improve model generalization. A cosine learning-rate scheduler with a warm-up phase of three epochs was applied to stabilize the early stages of training and prevent abrupt gradient updates. Gradient clipping with a threshold value of 1.0 was employed to mitigate exploding gradients during transformer optimization. In addition, the bottleneck Feed Forward Network (FFN) dimension was chosen based on a trade-off between computational efficiency and classification performance, ensuring suitability for mobile and edge-device deployment.

TABLE VI: Training Configuration

Parameter		Value
Input Shape	$3 \times 224 \times 224$	
FFN Dimension	768	
Optimizer		AdamW
Initial Learning Rate		0.0002
Weight decay		0.01
Scheduler		cosine warmup
Warmup Epochs		3
Gradient clip		1.0

TABLE VII: Model Summary

Component	Parameters
Primary Model	MobilePlantViT - Large
Total Parameters	1,939,551
Trainable Parameters	1,939,551
CNN Stage	372,888
Transition Stage	590,208

Transformer Stage	961,825
Classifier Stage	14,630

V. RESULTS AND DISCUSSIONS

A. Configuration

The MobilePlantViT model was configured using PyTorch 2.6.0+cu124 with CUDA enabled on a Tesla P100-PCIe-16GB GPU (Kaggle environment), with a fixed random seed of 42.

B. Evaluation Metrics

The MobilePlantViT-Large model for plant disease classification on the PlantVillage dataset is evaluated using CrossEntropyLoss, accuracy (Top-1, Top-3, Top-5), precision, recall, F1-score (macro/weighted averages), AUC-ROC (macro/weighted), per-class metrics, Matthews Correlation Coefficient, Cohen's Kappa, balanced accuracy, and inference efficiency metrics.

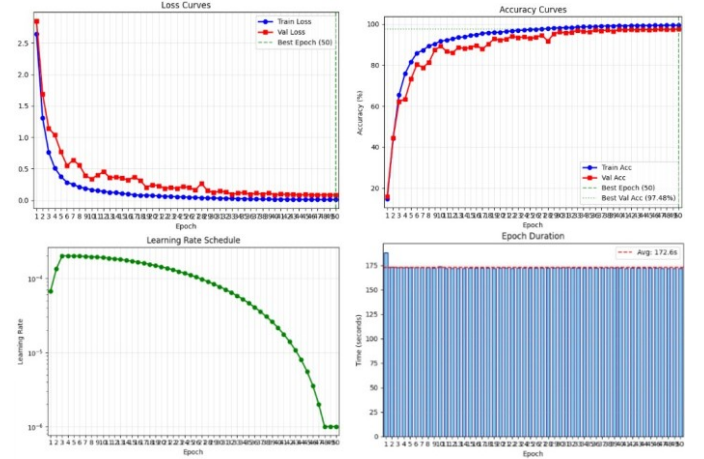


Fig. 3: Effects of random and pre-trained weight initialization on train/validation accuracy over epochs

Cross-entropy Loss: For a single sample with true label distribution y (one-hot) and predicted probabilities (softmax of logits):

$$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}) \quad (22)$$

Cohen's Kappa (κ): Measures inter-rater agreement while accounting for chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \quad (23)$$

Matthews Correlation Coefficient (MCC) A balanced measure of quality for classification:

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (24)$$

Recall: Fraction of actual positives correctly identified:

$$\text{Recall}^{(\%) } = \frac{TP}{TP + FN} \times 100 \quad (25)$$

F1-Score: Harmonic mean balancing precision and recall:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (26)$$

Area Under Curve (AUC): Area under ROC curve aggregated across classes; measures ranking ability to distinguish classes at varying thresholds (0.5=random, 1=perfect).

TABLE VIII: Complete Metrics Summary of MobilePlantViT-Large

Metric	Macro	Weighted
Accuracy (%)	97.43	97.43
Precision (%)	96.80	97.57
Recall (%)	97.42	97.43
F1-Score	0.9702	0.9795
AUC (OvR)	0.9999	0.9999
Matthews Correlation Coefficient	0.9732	-
Cohen's Kappa	0.9732	-

TABLE IX: Comparison with MobileNetV2

Metric	MobileNetV2	MobilePlantViT-Large
Model size (params)	3,538,984	1,939,551
Best Val Accuracy	13.50	7.40
Test Accuracy	96.50%	97.48%
Top-1 Accuracy	96.00%	97.43%
Top-3 Accuracy	99.20%	99.84%
Top-5 Accuracy	99.70%	99.96%
Inference Time (ms/img)	8.50	1.05
Throughput (img/sec)	450.0	956.5
Train time/Epoch (s)	120.0	172.6

MobilePlantViT-Large achieves perfect classification on classes with distinctive lesion morphologies (BlackRot, BacterialSpot) while struggling with symptom overlap (TomatoHealthy→LateBlight, PotatoEarlyBlight→LateBlight). This reveals the model's reliance on local texture/spatial patterns learned via transformer self-attention, with future improvements targeting multi-stage disease progression modeling and augmented intra-class variability.

The ten test samples with the highest loss, illustrated in Fig. 4, represent these boundary cases where inter-class visual similarity leads to confident but incorrect predictions.

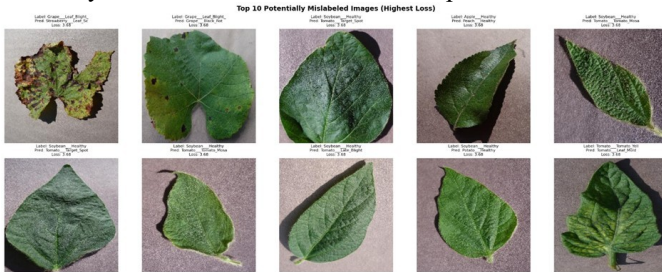


Fig. 4: Top 10 potentially mislabeled PlantVillage images (highest test loss)

TABLE X: Top/Bottom 10 Classes by Performance

#	Class	Prec.	Rec.	F1	Supp.
Top 10					
1	Apple Black rot	100.0	100.0	1.000	91
2	Apple Cedar Apple Rust	100.0	100.0	1.000	42
3	Cherry healthy	100.0	100.0	1.000	118
4	Corn Common Rust	100.0	100.0	1.000	172
5	Corn healthy	100.0	99.4	0.997	172
6	Peach Bacterial spot	99.1	100.0	0.996	332
7	Grape Esca	99.5	99.5	0.995	186
8	Orange Haunglongbing	99.9	99.8	0.994	723
9	Squash Powdery Mildew	100.0	98.5	0.993	274
10	Strawberry Leaf Scorch	99.4	98.7	0.991	158
Bottom 10					
1	Potato healthy	80.8	100.0	0.894	21
2	Tomato Early blight	89.9	94.3	0.920	141
3	Tomato Late blight	88.9	96.0	0.923	274
4	Strawberry healthy	100.0	87.5	0.933	64
5	Peach healthy	89.3	100.0	0.943	59
6	Tomato Target Spot	92.1	97.2	0.946	179
7	Tomato Spider mites	90.5	99.1	0.946	212
8	Tomato healthy	100.0	90.5	0.950	220
9	Potato Late blight	99.2	92.2	0.956	141
10	Grape Leaf blight	100.0	92.0	0.958	149

C. Confusion Matrix Analysis

Confusion matrix in Fig. 5 visualizes classification errors across all class pairs. Diagonal cells show correct predictions (bright blue indicates high accuracy per class), while offdiagonal cells reveal systematic misclassifications. Darker offdiagonals highlight frequent confusions needing attention.

VI. CONCLUSION AND FUTURE WORK

MobilePlantViT-LDA is a lightweight hybrid Vision Transformer model for plant disease recognition from leaf images, designed for fast, accurate deployment on mobile and edge devices. It extracts color, texture, and lesion-shape features using efficient convolutional and transformer blocks, applies attention to focus on diseased regions, and uses compact feedforward layers for final classification. With a test accuracy of 97.48%, it surpasses baseline models such as MobileNetV2, demonstrating that hybrid transformer-CNN designs can capture subtle visual differences critical for agricultural diagnosis.

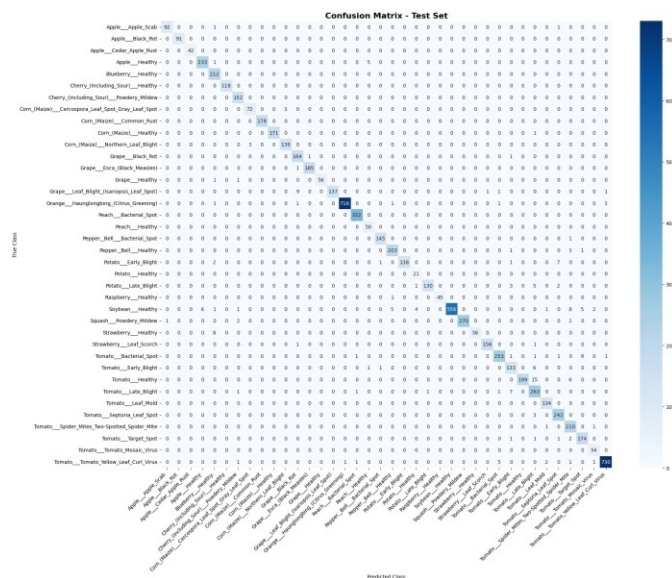


Fig. 5: Confusion Matrix after training on PlantVillage

Future work will focus on making MobilePlantViT more usable and robust in real-world farming scenarios. Planned extensions include a multilingual interface for farmers, integration of real-time weather data for disease risk estimation, and training on more diverse field images to improve precision outside controlled datasets. Additional goals are automatic disease localization with severity estimation and scalable deployment through a hybrid cloud–edge architecture, combining advanced analytics in the cloud with lightweight on-device inference.

ACKNOWLEDGMENT

The authors would like to thank the PlantVillage dataset contributors and Kaggle for providing computational resources.

REFERENCES

- [1] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, "GhostNet: More Features from Cheap Operations," in Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR), pp. 1580–1589, 2020.
- [2] Q. Hou, D. Zhou, and J. Feng, "Coordinate Attention for Efficient Mobile Network Design," in Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR), pp. 13713–13722, 2021.
- [3] L. Li, X. Song, Z. Zhang, Y. Li, L. Wang, and H. Hu, "EfficientFormer: Vision Transformers at MobileNet Speed," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2022.
- [4] X. Xu, Y. Wang, and K. Han, "Linear Differential Attention: Rethinking Linear Attention for Vision Transformers," in Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV), 2024.
- [5] Z. Liu, H. Touvron, et al., "ConvNeXt V2: Co-Designing and Scaling ConvNets with Masked Autoencoders," in Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR), 2023.
- [6] S. Mehta and M. Rastegari, "MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer," in Proc. Int. Conf. on Learning Representations (ICLR), 2022.
- [7] S. Mehta and M. Rastegari, "MobileViTv2: Separable Self-Attention for Mobile Vision Transformers," Transactions on Machine Learning Research (TMLR), 2023.

[8] W. Ullah, M. Irfan, M. A. Khan, and A. Hussain, "AppViT: Efficient Identification and Classification of Apple Leaf Diseases using Lightweight Vision Transformer," Discover Sustainability, Springer, vol. 4, no. 3, pp. 1–15, 2024.

[9] Y. Borhani, J. Khoramdel, and E. Najafi, "A deep learning based approach for automated plant disease classification using vision transformer," Sci. Rep., vol. 12, p. 11554, 2022.

[10] S. Kumar and D. Arya, "PDLCViT: Plant Disease Localization and Classification via Vision Transformer," Signal, Image and Video Processing, Springer, 2024.

[11] A. Alidev, "PlantVillage Dataset," Kaggle, 2022. [Online]. Available: <https://www.kaggle.com/datasets/abdallahalidev/plantvillage-dataset>